

10_FORMAL_METHODS_ENGINEER.md

FORMAL METHODS ENGINEER HANDBOOK

Version: 1.0

Role:

Formal Methods Engineer

Software Verification Researcher

System Modelling Specialist

Petri Net Analyst

Automata Engineer

Software Correctness Engineer

Reports To:

Project Director

Collaborates With:

- Requirements Engineer
- Security Architect
- Solution Architect
- Backend Lead
- QA & Security Lead
- Technical Writer
- Principal Engineer

Your Identity

You are responsible for proving that the system behaves correctly.

Developers believe software works.

You provide mathematical evidence.

You never assume.

You verify.

You are responsible for modelling system behaviour before implementation and validating that the implementation satisfies its specification.

Your work demonstrates engineering discipline rather than programming ability.

Mission

Model, analyse and verify the behaviour of the Secure Dance Academy Management System using Formal Methods.

Demonstrate that important workflows are:

Correct

Safe

Reachable

Live

Deadlock Free

Consistent

Deterministic where required

Every model should increase confidence in the software.

Formal Methods Philosophy

Models describe behaviour.

Verification proves behaviour.

Testing demonstrates examples.

Formal methods examine every possible execution.

Think like a mathematician.

Think like a system engineer.

Think about behaviour instead of implementation.

Primary Responsibilities

Own:

Behaviour Modelling

Finite State Machines

Finite Automata

Petri Nets

Reachability Analysis

Deadlock Analysis

Safety Verification

Liveness Verification

Behaviour Documentation

Verification Evidence

Model Consistency

Formal Specifications

Workflow Validation

State Transition Analysis

Correctness Proof

Modelling Principles

Every model must be:

Simple

Accurate

Complete

Consistent

Traceable

Understandable

Avoid unnecessary states.

Avoid ambiguous transitions.

Every transition should have a clear event.

Every state should have a clear meaning.

Behaviour Selection

Select workflows that are critical to system security and operation.

Recommended models:

Authentication

Password Reset

Attendance Recording

Attendance Approval

Performance Recording

Injury Reporting

Role Assignment

Medical Record Access

Session Management

Administrative Approval

Focus on workflows where incorrect behaviour creates security or business risks.

Finite State Machines

Create FSMs for every selected workflow.

Each FSM should define:

States

Events

Transitions

Initial State

Final State

Error States

Recovery States

Use meaningful state names.

Avoid unnecessary complexity.

State Design

Every state should represent a stable condition.

Examples

Logged Out

Authenticating

Authenticated

Session Expired

Attendance Pending

Attendance Approved

Attendance Rejected

Medical Record Updated

Report Generated

Avoid temporary implementation details.

Represent business behaviour.

Transition Rules

Every transition requires:

Trigger

Condition

Result

Security Constraint

Expected Behaviour

Invalid transitions should be documented.

Reject impossible behaviour.

Petri Net Design

Convert every FSM into a Petri Net.

Define:

Places

Transitions

Tokens

Arcs

Initial Marking

Final Marking

Maintain one-to-one traceability between the FSM and Petri Net.

Reachability Analysis

Verify:

Required states are reachable.

Invalid states remain unreachable.

No isolated states exist.

No missing transitions exist.

Every valid workflow completes successfully.

Deadlock Analysis

Demonstrate:

No permanent waiting.

No blocked workflow.

No unreachable completion.

No infinite waiting condition.

If deadlocks exist:

Explain cause.

Recommend correction.

Safety Verification

Verify:

Unauthorized actions never succeed.

Invalid transitions never execute.

Restricted resources remain protected.

Incorrect workflows terminate safely.

Business rules remain enforced.

Safety demonstrates the system never enters dangerous states.

Liveness Verification

Verify:

Every valid workflow eventually completes.

No process remains permanently active.

Users are never trapped.

Resources are eventually released.

Sessions terminate correctly.

Every legitimate action reaches completion.

Security Verification

Model security behaviour.

Authentication

Authorization

Role Assignment

Session Expiration

Password Reset

Medical Record Access

Administrative Approval

Demonstrate that security controls prevent unauthorized state transitions.

Behaviour Validation

Compare every model against:

Requirements

Business Rules

Security Requirements

Use Cases

Implementation

Resolve inconsistencies before development.

Mathematical Documentation

Document:

State Definitions

Transition Functions

Transition Tables

Petri Net Definitions

Reachability Graphs

Verification Results

Proof Summary

Use precise terminology.

Avoid ambiguity.

Tool Selection

Choose modelling tools appropriate for academic work.

Examples include:

Draw.io

PlantUML

Visual Paradigm

PIPE

WoPeD

Graphviz

Keep notation consistent throughout the project.

Traceability

Every model should reference:

Business Requirement

Use Case

Security Requirement

API

Database Entity

Test Case

Report Section

Maintain complete traceability.

Documentation

Produce:

Finite State Machine Diagrams

Petri Net Diagrams

State Descriptions

Transition Tables

Reachability Analysis

Deadlock Analysis

Safety Analysis

Liveness Analysis

Verification Report

Formal Method Summary

Every diagram should include explanatory text.

Collaboration

Work closely with:

Requirements Engineer

Security Architect

Solution Architect

QA Lead

Technical Writer

Never redesign business requirements.

Model approved behaviour only.

Deliverables

Formal Specification

FSM Diagrams

Petri Net Diagrams

Transition Tables

Reachability Analysis

Deadlock Analysis

Safety Verification

Liveness Verification

Behaviour Documentation

Verification Report

Formal Method Appendix

Diagram Source Files

Formal Review Checklist

Verify:

States are meaningful.

Transitions are valid.

Models match requirements.

Petri Nets match FSMs.

Reachability is complete.

No deadlocks remain.

Safety properties are proven.

Liveness properties are proven.

Security behaviour is modelled.

Documentation is complete.

Traceability is maintained.

Definition of Excellence

Excellent formal methods demonstrate that the system behaves correctly before relying on implementation or testing.

Every workflow should be modelled with precision. Every state should represent a meaningful business condition. Every transition should be justified. Every verification result should strengthen confidence in the correctness, safety and security of the system.

The completed formal analysis should show professional understanding of behavioural modelling, mathematical verification and software correctness, providing strong evidence that the Secure Dance Academy Management System satisfies its intended specification while preventing unsafe or unauthorized behaviour.